

Tarea 1

Aspectos Matemáticos y Computacionales del Aprendizaje Automático

28 de agosto de 2025

Fecha de entrega: Jueves 4 de septiembre, 2025.

1. Encuentra una transformación similar a la transformación min-max pero cuya imagen en cada coordenada sea el intervalo $[a, b]$ para algunos a, b dados.
2. Demuestra que para un conjunto de datos $A = \{x_1, x_2, \dots, x_n\} \subset \mathbb{R}^D$, el punto z que minimiza

$$f(z) = \sum_{i=1}^n \|x_i - z\|^2$$

es la media aritmética $\mu(A) = \frac{1}{n} \sum_{i=1}^n x_i$.

3. Sea $S^{D-1} \subset \mathbb{R}^D$ la esfera unitaria. Demuestra que la función $d_\theta : S^{D-1} \times S^{D-1} \rightarrow \mathbb{R}$ dada por

$$d_\theta(z, w) = \arccos \langle z, w \rangle$$

es una métrica en S^{D-1} . Esta es la **métrica angular**.

En una notebook de Google Colab realiza los siguientes ejercicios y experimentos. Cada ejercicio irá en una sección separada.

1. Realiza la reducción de dimensionalidad descrita en el ejercicio 4 de la [notebook 1](#).
2. Comprueba las cotas de la distorsión de las distancias Euclidianas en las transformaciones min-max y z-score de la proposición vista en clase. Para esto:
 - a) Genera un conjunto A de 100 puntos con una distribución uniforme en el intervalo $[1, 10]$.
 - b) Aplica las transformaciones F (min-max) y G (z-score) a cada punto de A (usa las implementaciones de scikit-learn usadas en la [notebook 3](#)) para tener los conjuntos transformados $F(A)$ y $G(A)$.
 - c) Calcula las constantes de distorsión para F y G .

- d) Escoge parejas de puntos diferentes aleatorias en A y verifica las desigualdades

$$\min_{1 \leq i \leq D} |a_i| \cdot d(x, y) \leq d(F(x), F(y)) \leq \max_{1 \leq i \leq D} |a_i| \cdot d(x, y)$$

analogamente con G .

3. Entrena un modelo de aprendizaje automático para predecir el precio de las casas usando el dataset *California Housing*, puedes consultar la notebook para leer el dataset. Realiza lo siguiente:
- a) Aplica un método de re-escalamiento a los datos para generar dos versiones del dataset: la original y la re-escalada. **Cuidado con el data leakage.**
 - b) Con cada versiones entrena un modelo de vecinos más cercanos. Es una tarea de regresión por lo que usaremos la clase `KNeighborsRegressor`.
 - c) Usando el score de rendimiento `mean_absolute_error`, compara el desempeño del modelo en cada versión del conjunto de prueba.